Shopify Product Type Gallery App

Build a Shopify embedded app that allows merchants to assign additional image and video galleries to products based on configurable conditions.

The goal is to replace an existing Metaobject-based implementation with a scalable app-based solution that works efficiently on stores with 5,000+ products.

Tech Stack

Use:

- Shopify App Remix template
- Polaris
- Shopify Admin GraphQL API
- Theme App Extension
- App Embed Block
- Prisma + SQLite for development
- Prisma-compatible schema designed for future migration to PostgreSQL

The app must not depend on Shopify Metaobjects.

Main Concept

The app allows merchants to create Gallery Rules.

Each Gallery Rule contains:

- Images
- Videos
- Conditions

When a product matches a rule, the images and videos are appended to the end of the product media gallery.

Data Model

GalleryRule

Fields:

- id
- name
- priority (integer)
- createdAt
- updatedAt

Relations:

- hasMany GalleryMedia
- hasMany GalleryCondition

GalleryMedia

Fields:

- id
- galleryRuleId
- mediaType (image | video)
- shopifyFileId
- url
- sortOrder

Media must preserve merchant ordering.

GalleryCondition

Fields:

- id
- galleryRuleId
- productType (required)
- titleContains (optional)
- tagIds (multiple optional)

Rules:

- Product Type is mandatory
- Title Contains is optional

- Tags are optional

Condition Logic

A product matches a condition when:

1. Product Type equals condition.productType

AND

1. If titleContains is populated: product.title contains titleContains

AND

1. If tags are selected: product contains ALL selected tags

Example:

Condition:

- Product Type = Sleeve
- Title Contains = Pro
- Tags = Football, Premium

Product matches only if:

- product.type == Sleeve
- title contains "Pro"
- has Football tag
- has Premium tag

Rule Logic

A Gallery Rule may contain multiple conditions.

Rule matches if ANY condition matches.

Pseudo:

matchRule = condition1 OR condition2 OR condition3

Within each condition:

productType AND titleContains AND tags

Admin Interface

Create a page:

Gallery Rules

Table columns:

- Name
- Conditions Count
- Images Count
- Videos Count
- Priority
- Edit
- Delete

Create/Edit Gallery Rule

Sections:

Rule Details

- Name
- Priority

Media

Images uploader:

- Select one or many Shopify files
- Drag and drop ordering

Videos uploader:

- Select one or many Shopify hosted videos
- Drag and drop ordering

Store Shopify file IDs whenever possible.

Conditions

Allow multiple conditions.

Each condition contains:

Product Type

Dropdown.

Load all distinct product types currently existing in the store.

Use GraphQL queries.

Required.

Title Contains

Text input.

Optional.

Case-insensitive matching.

Product Tags

Multi-select.

Load all tags existing in the store.

Searchable.

Optional.

Condition examples:

Condition A:

Product Type = Sleeve Title Contains = Pro

Condition B:

Product Type = Spats

Condition C:

Product Type = Visor Tag = Football

Product Matching Service

Create a reusable service:

evaluateGalleryRules(product)

Returns:

{ images: [], videos: [] }

Process:

1. Load rules ordered by priority DESC
2. Evaluate conditions
3. Merge matching media
4. Remove duplicates
5. Preserve ordering

Storefront Integration

Create a Theme App Extension.

The extension must expose a block that can be enabled from Theme Customizer.

The block must:

1. Detect current product
2. Request matching gallery media from app endpoint
3. Return:

{ images: [], videos: [] }

1. Append media after native Shopify product media

Do not replace existing media.

Append only.

API Endpoint

Create endpoint:

GET

/apps/gallery-media

Parameters:

productId

Returns:

{ images: [], videos: [] }

Response example:

{ "images": [{ "id": "...", "url": "..." }], "videos": [{ "id": "...", "url": "..." }] }

Theme Integration Requirements

The current theme already has custom gallery rendering.

The extension must provide data only.

Do not force a specific frontend gallery implementation.

Return media in a format easily consumable by Liquid or JavaScript.

Provide examples for:

- Main gallery rendering
- Thumbnail rendering
- Video rendering

so existing code can be adapted.

Performance Requirements

Store may contain:

- 5,000+ products
- Hundreds of gallery rules

Requirements:

- No full product scans
- Evaluate only current product
- Cache rule data
- Minimize GraphQL requests
- Use indexed database fields

Target response time:

< 100ms after cache warmup

Future Requirements

Design schema so future conditions can be added without major refactoring:

Potential future conditions:

- Vendor
- Product Handle
- Product Category
- Collections
- SKU contains
- Variant options

Condition engine should be extensible.

Deliverables

Generate:

1. Prisma schema
2. Database migrations
3. Remix routes
4. GraphQL services
5. Polaris UI

6. Theme App Extension
7. Rule evaluation service
8. API endpoint
9. Installation instructions
10. Seed data for testing

Follow Shopify app best practices and production-ready architecture.